



**UNIVERSIDAD PRIVADA DE TACNA**

**FACULTAD DE INGENIERÍA**

**Escuela Profesional de Ingeniería de Sistemas**

**Proyecto DataQuest:**

***Validador de Normalización de Bases de Datos Relacionales***

*Curso: Base de Datos II*

*Docente: Ing. Patrick Jose Cuadros Quiroga*

Integrantes:

**Dongo Palza, Manuel Andre** **(2023076802)**

**Perez Peralta, Fabrizio Salvador** **(2023077476)**

**Tacna - Perú**  
**2026**

## CONTROL DE VERSIONES

| Versión | Hecha por | Revisada por          | Aprobada por | Fecha      | Motivo                                    |
|---------|-----------|-----------------------|--------------|------------|-------------------------------------------|
| 1.0     | MADP      | Pendiente de revisión | Pendiente    | 04/07/2026 | Versión inicial del diccionario de datos. |

### Contenido

|                                                       |    |
|-------------------------------------------------------|----|
| CONTROL DE VERSIONES .....                            | 2  |
| DICCIONARIO DE DATOS .....                            | 4  |
| 1. Modelo Entidad / Relación .....                    | 4  |
| 1.1. Descripción general del modelo .....             | 4  |
| 1.2. Entidades principales .....                      | 4  |
| 1.3. Relaciones del sistema .....                     | 4  |
| 1.4. Diagrama entidad-relación .....                  | 5  |
| 2. Diseño Lógico.....                                 | 5  |
| 2.1. Modelo lógico de datos .....                     | 5  |
| 2.2. Entidades recomendadas futuras .....             | 5  |
| 2.3. Reglas lógicas del modelo .....                  | 6  |
| 3. Diseño Físico.....                                 | 6  |
| 3.1. Motor de base de datos.....                      | 6  |
| 3.2. Esquema físico consolidado.....                  | 6  |
| 3.3. Tipos de datos utilizados .....                  | 6  |
| 3.4. Claves primarias, foráneas y restricciones ..... | 6  |
| 3.5. Uso de JSONB.....                                | 7  |
| 4. Diccionario de Datos .....                         | 7  |
| 4.1. Tabla usuarios .....                             | 7  |
| 4.2. Tabla perfiles .....                             | 7  |
| 4.3. Tabla presencia .....                            | 8  |
| 4.4. Tabla historial_validaciones .....               | 8  |
| 4.5. Tablas recomendadas futuras .....                | 9  |
| 5. Procedimientos Almacenados .....                   | 9  |
| 5.1. Parámetros recomendados de procedimientos.....   | 10 |
| 5.2. Funciones y triggers recomendados.....           | 10 |
| 6. Lenguaje de Definición de Datos (DDL) .....        | 10 |
| 7. Lenguaje de Manipulación de Datos (DML) .....      | 11 |

|                                          |    |
|------------------------------------------|----|
| 8. Reglas de Integridad y Seguridad..... | 12 |
| 9. Matriz de Trazabilidad de Datos ..... | 12 |
| 10. Conclusiones Técnicas.....           | 13 |
| 11. Recomendaciones .....                | 13 |

## DICCIONARIO DE DATOS

El presente documento describe el diccionario de datos del sistema DataQuest: Validador de Normalización de Bases de Datos Relacionales. La finalidad del documento es registrar formalmente la estructura de datos utilizada por el sistema, sus entidades, tablas, campos, relaciones, restricciones, reglas de integridad, scripts de definición y ejemplos de manipulación de datos.

El modelo documentado corresponde al esquema real proporcionado para PostgreSQL/Supabase, compuesto por cuatro tablas implementadas: usuarios, perfiles, presencia e historial\_validaciones. Las tablas adicionales que podrían fortalecer la evolución del sistema se registran únicamente como recomendadas o futuras, sin clasificarlas como implementadas.

## 1. Modelo Entidad / Relación

### 1.1. Descripción general del modelo

El modelo entidad/relación de DataQuest se orienta a registrar usuarios académicos, datos complementarios de perfil, estado de presencia y el historial de validaciones ejecutadas sobre esquemas relacionales. La tabla central es usuarios, desde la cual se desprenden relaciones con perfiles, presencia e historial\_validaciones.

La entidad historial\_validaciones concentra la evidencia principal del sistema: cada análisis de normalización puede almacenar el esquema original, el esquema final, las dependencias funcionales, las violaciones detectadas y las sugerencias generadas. Estos componentes se almacenan como JSONB para conservar estructuras flexibles producidas por el motor Python del validador.

### 1.2. Entidades principales

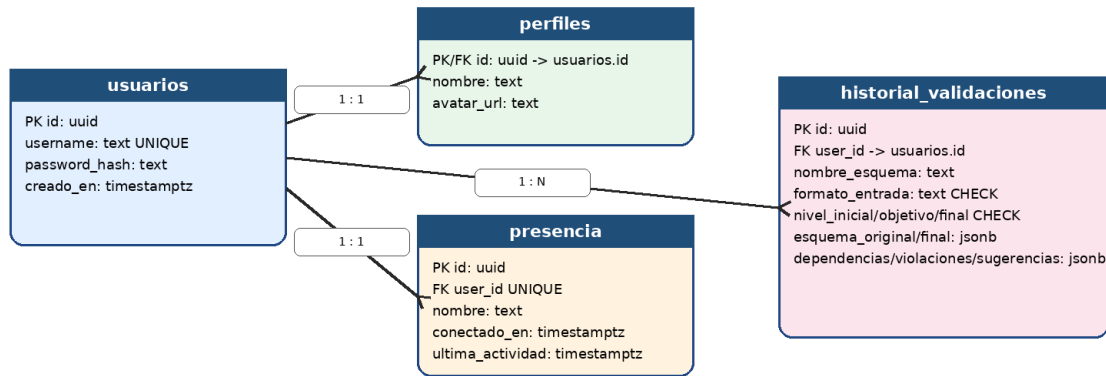
| Entidad              | Tabla física           | Descripción                                                                                          | Estado       |
|----------------------|------------------------|------------------------------------------------------------------------------------------------------|--------------|
| USUARIO              | usuarios               | Persona registrada en DataQuest. Permite asociar validaciones a una cuenta y conservar trazabilidad. | Implementada |
| PERFIL               | perfiles               | Información complementaria del usuario, como nombre visible y avatar.                                | Implementada |
| PRESENCIA            | presencia              | Estado de conexión y última actividad de un usuario en la comunidad o entorno colaborativo.          | Implementada |
| HISTORIAL_VALIDACION | historial_validaciones | Registro histórico de cada validación de normalización ejecutada por el usuario.                     | Implementada |

### 1.3. Relaciones del sistema

| Relación                          | Cardinalidad | Clave foránea                                 | Descripción                                                             |
|-----------------------------------|--------------|-----------------------------------------------|-------------------------------------------------------------------------|
| usuarios - perfiles               | 1 a 1        | perfiles.id -> usuarios.id                    | Cada usuario puede tener un perfil asociado.                            |
| usuarios - presencia              | 1 a 1        | presencia.user_id -> usuarios.id              | Cada usuario puede tener un registro de presencia activo o actualizado. |
| usuarios - historial_validaciones | 1 a N        | historial_validaciones.user_id -> usuarios.id | Un usuario puede ejecutar muchas validaciones a lo largo del tiempo.    |

## 1.4. Diagrama entidad-relación

Diagrama Entidad-Relación DataQuest



Fuente: elaboración propia basada en el esquema real Supabase/PostgreSQL de DataQuest.

Figura 1. Diagrama entidad-relación de DataQuest. Fuente: elaboración propia.

## 2. Diseño Lógico

### 2.1. Modelo lógico de datos

El diseño lógico traduce las necesidades del sistema en entidades y relaciones independientes del motor físico. En DataQuest, las entidades implementadas se concentran en la autenticación básica, el perfil visible, la presencia del usuario y el almacenamiento del historial de validaciones.

| Entidad              | Descripción                                                      | Atributos principales                                                           | Identificador lógico | Relaciones                              | Cardinalidad  | Estado       |
|----------------------|------------------------------------------------------------------|---------------------------------------------------------------------------------|----------------------|-----------------------------------------|---------------|--------------|
| USUARIO              | Representa a cada usuario registrado en el sistema.              | id, username, password_hash, creado_en                                          | id                   | PERFIL, PRESENCIA, HISTORIAL_VALIDACION | 1:1, 1:1, 1:N | Implementada |
| PERFIL               | Representa información complementaria y visual del usuario.      | id, nombre, avatar_url                                                          | id                   | USUARIO                                 | 1:1           | Implementada |
| PRESENCIA            | Representa el estado de conexión y última actividad.             | id, user_id, nombre, conectado_en, ultima_actividad                             | id                   | USUARIO                                 | 1:1           | Implementada |
| HISTORIAL_VALIDACION | Representa una validación ejecutada sobre un esquema relacional. | id, user_id, fecha, nombre_esquema, formato_entrada, niveles, jsonb de análisis | id                   | USUARIO                                 | N:1           | Implementada |

### 2.2. Entidades recomendadas futuras

Las siguientes entidades no forman parte del esquema implementado. Se sugieren únicamente como ampliaciones futuras si el proyecto requiere mayor normalización de resultados, auditoría avanzada o separación de reportes.

| Entidad recomendada | Propósito                                                 | Motivo de recomendación                                       | Estado                        |
|---------------------|-----------------------------------------------------------|---------------------------------------------------------------|-------------------------------|
| REPORTE             | Registrar reportes exportados o generados por el sistema. | Permitir trazabilidad de archivos PDF/TXT/SQL generados.      | No implementada / Recomendada |
| AUDITORIA_EVENTO    | Registrar eventos técnicos y de seguridad.                | Mejorar trazabilidad de accesos, inserciones y eliminaciones. | No implementada / Recomendada |

| Entidad recomendada             | Propósito                                                   | Motivo de recomendación                              | Estado                        |
|---------------------------------|-------------------------------------------------------------|------------------------------------------------------|-------------------------------|
| CONFIGURACION_SISTEMA           | Guardar parámetros generales del sistema.                   | Evitar configuración dispersa en código.             | No implementada / Recomendada |
| RESULTADO_DETALLADO             | Separar resultados de normalización por nivel.              | Permitir consultas analíticas sin descomponer JSONB. | No implementada / Recomendada |
| DEPENDENCIA_FUNCIONAL_DETALLADA | Registrar dependencias funcionales como filas normalizadas. | Mejorar análisis formal y reportes relacionales.     | No implementada / Recomendada |

## 2.3. Reglas lógicas del modelo

- Todo usuario se identifica mediante un UUID único.
- El nombre de usuario debe ser único para evitar duplicidad de cuentas.
- Un perfil debe pertenecer a un usuario existente.
- Un registro de presencia debe pertenecer a un usuario existente y no debe duplicarse por usuario.
- Una validación histórica puede estar asociada a un usuario y debe registrar el nombre del esquema analizado.
- Los niveles de normalización se restringen a valores permitidos para evitar datos inconsistentes.
- Los detalles dinámicos del análisis se almacenan en JSONB para conservar flexibilidad.

## 3. Diseño Físico

### 3.1. Motor de base de datos

El diseño físico está orientado a PostgreSQL en Supabase. El esquema utiliza el espacio de nombres public y emplea UUID como identificador principal. Este enfoque permite generar claves globalmente únicas, adecuadas para ambientes web, API y servicios conectados.

### 3.2. Esquema físico consolidado

| Tabla física           | Motor / esquema     | Tipo de clave                | Relaciones principales            | Estado       |
|------------------------|---------------------|------------------------------|-----------------------------------|--------------|
| usuarios               | PostgreSQL / public | UUID con gen_random_uuid()   | Tabla central del modelo          | Implementada |
| perfiles               | PostgreSQL / public | UUID heredado de usuarios.id | FK a usuarios(id)                 | Implementada |
| presencia              | PostgreSQL / public | UUID con gen_random_uuid()   | FK a usuarios(id), user_id UNIQUE | Implementada |
| historial_validaciones | PostgreSQL / public | UUID con gen_random_uuid()   | FK a usuarios(id), campos JSONB   | Implementada |

### 3.3. Tipos de datos utilizados

| Tipo de dato             | Uso en DataQuest                                                     | Justificación técnica                                                                                 |
|--------------------------|----------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| uuid                     | Identificadores primarios y foráneos.                                | Permite identificación única sin depender de secuencias locales.                                      |
| text                     | Campos variables como username, nombre, avatar_url y nombre_esquema. | Adecuado para textos de longitud variable en PostgreSQL.                                              |
| timestamp with time zone | Fechas de creación, conexión, actividad y validación.                | Conserva contexto temporal y zona horaria.                                                            |
| jsonb                    | Esquemas, dependencias, violaciones y sugerencias.                   | Permite almacenar estructuras flexibles y consultar componentes JSON si se requieren índices futuros. |
| CHECK                    | Dominios de formato y niveles de normalización.                      | Reduce errores de datos y restringe valores permitidos.                                               |

### 3.4. Claves primarias, foráneas y restricciones

| Tabla                  | Clave primaria | Claves foráneas         | Restricciones relevantes                                    |
|------------------------|----------------|-------------------------|-------------------------------------------------------------|
| usuarios               | id             | No aplica               | username UNIQUE, password_hash NOT NULL                     |
| perfiles               | id             | id -> usuarios(id)      | Relación 1 a 1 con usuarios                                 |
| presencia              | id             | user_id -> usuarios(id) | user_id UNIQUE                                              |
| historial_validaciones | id             | user_id -> usuarios(id) | nombre_esquema NOT NULL; CHECK en formato_entrada y niveles |

### 3.5. Uso de JSONB

La tabla historial\_validaciones utiliza columnas JSONB para almacenar estructuras de salida del motor de normalización. Esta decisión evita crear un conjunto excesivo de tablas físicas para cada detalle del análisis y permite registrar datos variables como dependencias, violaciones o sugerencias, los cuales pueden cambiar según la complejidad del esquema ingresado por el usuario.

## 4. Diccionario de Datos

En esta sección se documentan únicamente las tablas implementadas en el esquema real de DataQuest. Las fichas siguen la estructura solicitada por la plantilla institucional: nombre de tabla, descripción, objetivo, relaciones y detalle de campos.

### 4.1. Tabla usuarios

| Elemento                    | Detalle                                                                                  |
|-----------------------------|------------------------------------------------------------------------------------------|
| Nombre de la Tabla          | usuarios                                                                                 |
| Descripción de la Tabla     | Contiene las credenciales básicas de los usuarios registrados en DataQuest.              |
| Objetivo                    | Permitir la identificación de usuarios y asociar cada validación a una cuenta.           |
| Relaciones con otras Tablas | perfiles.id, presencia.user_id e historial_validaciones.user_id referencian usuarios.id. |
| Estado                      | Implementada                                                                             |
| Módulo asociado             | Autenticación, gestión de sesión, historial de validaciones.                             |

#### Descripción de los campos

| Nro. | Nombre del campo | Tipo dato                | Longitud | Permite nulos | Clave primaria | Clave foránea | Valor por defecto | Restricciones    | Descripción del campo                                                         |
|------|------------------|--------------------------|----------|---------------|----------------|---------------|-------------------|------------------|-------------------------------------------------------------------------------|
| 1    | id               | uuid                     | 36       | No            | Sí             | No            | gen_random_uuid() | PRIMARY KEY      | Identificador único del usuario.                                              |
| 2    | username         | text                     | Variable | No            | No             | No            | No aplica         | UNIQUE, NOT NULL | Nombre de usuario utilizado para el acceso o identificación.                  |
| 3    | password_hash    | text                     | Variable | No            | No             | No            | No aplica         | NOT NULL         | Contraseña protegida mediante hash; no almacena la contraseña en texto plano. |
| 4    | creado_en        | timestamp with time zone | -        | Sí            | No             | No            | now()             | DEFAULT          | Fecha y hora de creación del usuario.                                         |

### 4.2. Tabla perfiles

| Elemento                    | Detalle                                                                                  |
|-----------------------------|------------------------------------------------------------------------------------------|
| Nombre de la Tabla          | perfiles                                                                                 |
| Descripción de la Tabla     | Contiene información complementaria del usuario, separada de los datos de autenticación. |
| Objetivo                    | Registrar nombre visible y avatar del usuario sin modificar la tabla de credenciales.    |
| Relaciones con otras Tablas | perfiles.id referencia usuarios.id.                                                      |
| Estado                      | Implementada                                                                             |
| Módulo asociado             | Perfil de usuario, vista de comunidad y personalización básica.                          |

#### Descripción de los campos

| Nro. | Nombre del campo | Tipo dato | Longitud | Permite nulos | Clave primaria | Clave foránea | Valor por defecto | Restricciones             | Descripción del campo                                     |
|------|------------------|-----------|----------|---------------|----------------|---------------|-------------------|---------------------------|-----------------------------------------------------------|
| 1    | id               | uuid      | 36       | No            | Sí             | Sí            | No aplica         | PRIMARY KEY, FOREIGN KEY  | Identificador del perfil y referencia directa al usuario. |
| 2    | nombre           | text      | Variable | Sí            | No             | No            | No aplica         | Sin restricción adicional | Nombre visible del                                        |

| Nro. | Nombre del campo | Tipo dato | Longitud | Permite nulos | Clave primaria | Clave foránea | Valor por defecto | Restricciones | Descripción del campo                     |
|------|------------------|-----------|----------|---------------|----------------|---------------|-------------------|---------------|-------------------------------------------|
|      |                  |           |          |               |                |               |                   |               | usuario dentro de DataQuest.              |
| 3    | avatar_url       | text      | Variable | Sí            | No             | No            | No aplica         | URL opcional  | Dirección de imagen o avatar del usuario. |

#### 4.3. Tabla presencia

| Elemento                    | Detalle                                                                 |
|-----------------------------|-------------------------------------------------------------------------|
| Nombre de la Tabla          | presencia                                                               |
| Descripción de la Tabla     | Registra el estado de conexión y última actividad de un usuario.        |
| Objetivo                    | Permitir control básico de usuarios conectados o activos en el sistema. |
| Relaciones con otras Tablas | presencia.user_id referencia usuarios.id.                               |
| Estado                      | Implementada                                                            |
| Módulo asociado             | Comunidad, presencia en tiempo real, monitoreo de actividad.            |

#### Descripción de los campos

| Nro. | Nombre del campo | Tipo dato                | Longitud | Permite nulos | Clave primaria | Clave foránea | Valor por defecto | Restricciones             | Descripción del campo                                                                                 |
|------|------------------|--------------------------|----------|---------------|----------------|---------------|-------------------|---------------------------|-------------------------------------------------------------------------------------------------------|
| 1    | id               | uuid                     | 36       | No            | Sí             | No            | gen_random_uuid() | PRIMARY KEY               | Identificador único del registro de presencia.                                                        |
| 2    | user_id          | uuid                     | 36       | Sí            | No             | Sí            | No aplica         | UNIQUE, FOREIGN KEY       | Usuario asociado a la presencia. Debe ser único para evitar múltiples presencias activas por usuario. |
| 3    | nombre           | text                     | Variable | Sí            | No             | No            | No aplica         | Sin restricción adicional | Nombre visible registrado para la presencia.                                                          |
| 4    | conectado_en     | timestamp with time zone | -        | Sí            | No             | No            | now()             | DEFAULT                   | Fecha y hora de conexión inicial.                                                                     |
| 5    | ultima_actividad | timestamp with time zone | -        | Sí            | No             | No            | now()             | DEFAULT                   | Última actividad registrada del usuario.                                                              |

#### 4.4. Tabla historial\_validaciones

| Elemento                    | Detalle                                                                                                                      |
|-----------------------------|------------------------------------------------------------------------------------------------------------------------------|
| Nombre de la Tabla          | historial_validaciones                                                                                                       |
| Descripción de la Tabla     | Almacena cada validación realizada en DataQuest.                                                                             |
| Objetivo                    | Conservar evidencia histórica de los esquemas analizados, niveles de normalización, dependencias, violaciones y sugerencias. |
| Relaciones con otras Tablas | historial_validaciones.user_id referencia usuarios.id.                                                                       |
| Estado                      | Implementada                                                                                                                 |
| Módulo asociado             | Validador, historial, diagnóstico, reporte académico.                                                                        |

#### Descripción de los campos

| Nro. | Nombre del campo | Tipo dato                | Longitud | Permite nulos | Clave primaria | Clave foránea | Valor por defecto | Restricciones | Descripción del campo                                |
|------|------------------|--------------------------|----------|---------------|----------------|---------------|-------------------|---------------|------------------------------------------------------|
| 1    | id               | uuid                     | 36       | No            | Sí             | No            | gen_random_uuid() | PRIMARY KEY   | Identificador único de la validación registrada.     |
| 2    | user_id          | uuid                     | 36       | Sí            | No             | Sí            | No aplica         | FOREIGN KEY   | Usuario que ejecutó la validación.                   |
| 3    | fecha            | timestamp with time zone | -        | Sí            | No             | No            | now()             | DEFAULT       | Fecha y hora de ejecución de la validación.          |
| 4    | nombre_esquema   | text                     | Variable | No            | No             | No            | No aplica         | NOT NULL      | Nombre asignado por el usuario al esquema analizado. |



| Nro. | Nombre del campo | Tipo dato | Longitud | Permite nulos | Clave primaria | Clave foránea | Valor por defecto | Restricciones | Descripción del campo                                                         |
|------|------------------|-----------|----------|---------------|----------------|---------------|-------------------|---------------|-------------------------------------------------------------------------------|
| 5    | formato_entrada  | text      | Variable | Sí            | No             | No            | No aplica         | CHECK         | Formato del archivo o contenido ingresado: sql, csv, xls, txt o texto pegado. |
| 6    | nivel_inicial    | text      | Variable | Sí            | No             | No            | No aplica         | CHECK         | Nivel inicial detectado antes de aplicar sugerencias.                         |
| 7    | nivel_objetivo   | text      | Variable | Sí            | No             | No            | No aplica         | CHECK         | Nivel de normalización que el usuario desea alcanzar.                         |
| 8    | nivel_final      | text      | Variable | Sí            | No             | No            | No aplica         | CHECK         | Nivel final alcanzado o estimado después del análisis.                        |
| 9    | esquema_original | jsonb     | Variable | Sí            | No             | No            | No aplica         | JSONB         | Representación estructurada del esquema antes de normalizar.                  |
| 10   | esquema_final    | jsonb     | Variable | Sí            | No             | No            | No aplica         | JSONB         | Representación estructurada posterior a sugerencias o correcciones.           |
| 11   | dependencias     | jsonb     | Variable | Sí            | No             | No            | No aplica         | JSONB         | Dependencias funcionales detectadas o registradas.                            |
| 12   | violaciones      | jsonb     | Variable | Sí            | No             | No            | No aplica         | JSONB         | Violaciones de normalización detectadas por el motor.                         |
| 13   | sugerencias      | jsonb     | Variable | Sí            | No             | No            | No aplica         | JSONB         | Sugerencias generadas por el sistema para mejorar el esquema.                 |

#### 4.5. Tablas recomendadas futuras

| Tabla recomendada     | Propósito                                         | Motivo                                                 | Estado                        |
|-----------------------|---------------------------------------------------|--------------------------------------------------------|-------------------------------|
| reportes              | Almacenar reportes generados por el usuario.      | Permitir trazabilidad de exportaciones PDF, TXT o SQL. | Recomendada / No implementada |
| auditoria_eventos     | Registrar eventos técnicos y de seguridad.        | Mejorar trazabilidad de acciones críticas.             | Recomendada / No implementada |
| configuracion_sistema | Guardar parámetros generales del sistema.         | Evitar configuración dispersa en código.               | Recomendada / No implementada |
| resultados_detallados | Normalizar resultados por nivel de normalización. | Facilitar consultas analíticas sin descomponer JSONB.  | Recomendada / No implementada |

### 5. Procedimientos Almacenados

En el esquema proporcionado no se evidencian procedimientos almacenados implementados. DataQuest opera principalmente desde la capa de aplicación, utilizando Streamlit, controladores Python, FastAPI y Supabase/PostgreSQL. Por ello, los siguientes procedimientos se presentan únicamente como recomendaciones futuras para fortalecer la base de datos.

| Nombre del SP                   | Descripción del procedimiento                               | Estado      | Relaciones con tablas            |
|---------------------------------|-------------------------------------------------------------|-------------|----------------------------------|
| sp_registrar_validacion         | Registra una validación completa en historial_validaciones. | Recomendado | historial_validaciones           |
| sp_obtener_historial_usuario    | Obtiene el historial filtrado por usuario y fecha.          | Recomendado | usuarios, historial_validaciones |
| sp_actualizar_presencia_usuario | Actualiza última actividad y nombre de presencia.           | Recomendado | presencia                        |
| sp_obtener_resumen_validaciones | Genera resumen de validaciones por nivel final y formato.   | Recomendado | historial_validaciones           |
| sp_eliminar_historial_antiguo   | Elimina o archiva validaciones antiguas de prueba.          | Recomendado | historial_validaciones           |

## 5.1. Parámetros recomendados de procedimientos

| SP                              | Parámetro        | Tipo dato | IN | OUT | Descripción                            |
|---------------------------------|------------------|-----------|----|-----|----------------------------------------|
| sp_registrar_validacion         | p_user_id        | uuid      | Sí | No  | Usuario que realiza la validación.     |
| sp_registrar_validacion         | p_nombre_esquema | text      | Sí | No  | Nombre del esquema validado.           |
| sp_registrar_validacion         | p_resultado_id   | uuid      | No | Sí  | ID de la validación creada.            |
| sp_obtener_historial_usuario    | p_user_id        | uuid      | Sí | No  | Usuario cuyo historial se consulta.    |
| sp_actualizar_presencia_usuario | p_user_id        | uuid      | Sí | No  | Usuario cuya actividad se actualizará. |
| sp_obtener_resumen_validaciones | p_user_id        | uuid      | Sí | No  | Usuario para el cálculo del resumen.   |

## 5.2. Funciones y triggers recomendados

| Elemento                        | Tipo    | Objetivo                                                          | Estado      |
|---------------------------------|---------|-------------------------------------------------------------------|-------------|
| fn_total_validaciones_usuario   | Función | Calcular el total de validaciones registradas por usuario.        | Recomendada |
| fn_validaciones_por_nivel_final | Función | Agrupar validaciones por nivel final alcanzado.                   | Recomendada |
| fn_resumen_formatos_entrada     | Función | Resumir validaciones según formato de entrada.                    | Recomendada |
| trg_actualizar_ultima_actividad | Trigger | Actualizar presencia cuando el usuario interactúa con el sistema. | Recomendado |
| trg_auditar_insert_historial    | Trigger | Registrar auditoría al insertar historial de validaciones.        | Recomendado |

## 6. Lenguaje de Definición de Datos (DDL)

El siguiente DDL está organizado para PostgreSQL/Supabase y respeta las cuatro tablas implementadas indicadas en el esquema proporcionado. Se incluyen índices recomendados para mejorar consultas frecuentes por usuario, fecha, nivel final y última actividad.

```
CREATE EXTENSION IF NOT EXISTS pgcrypto;

CREATE TABLE IF NOT EXISTS public.usuarios (
  id uuid NOT NULL DEFAULT gen_random_uuid(),
  username text NOT NULL UNIQUE,
  password_hash text NOT NULL,
  creado_en timestamp with time zone DEFAULT now(),
  CONSTRAINT usuarios_pkey PRIMARY KEY (id)
);

CREATE TABLE IF NOT EXISTS public.perfiles (
  id uuid NOT NULL,
  nombre text,
  avatar_url text,
  CONSTRAINT perfiles_pkey PRIMARY KEY (id),
  CONSTRAINT perfiles_id_fkey FOREIGN KEY (id)
    REFERENCES public.usuarios(id)
);

CREATE TABLE IF NOT EXISTS public.presencia (
  id uuid NOT NULL DEFAULT gen_random_uuid(),
  user_id uuid UNIQUE,
  nombre text,
  conectado_en timestamp with time zone DEFAULT now(),
  ultima_actividad timestamp with time zone DEFAULT now(),
  CONSTRAINT presencia_pkey PRIMARY KEY (id),
  CONSTRAINT presencia_user_id_fkey FOREIGN KEY (user_id)
    REFERENCES public.usuarios(id)
```

```

);

CREATE TABLE IF NOT EXISTS public.historial_validaciones (
  id uuid NOT NULL DEFAULT gen_random_uuid(),
  user_id uuid,
  fecha timestamp with time zone DEFAULT now(),
  nombre_esquema text NOT NULL,
  formato_entrada text CHECK (formato_entrada = ANY
    (ARRAY['sql'::text, 'csv'::text, 'xls'::text, 'txt'::text, 'texto_pegado'::text])),
  nivel_inicial text CHECK (nivel_inicial = ANY
    (ARRAY['sin_normalizar'::text, '1fn'::text, '2fn'::text, '3fn'::text])),
  nivel_objetivo text CHECK (nivel_objetivo = ANY
    (ARRAY['1fn'::text, '2fn'::text, '3fn'::text])),
  nivel_final text CHECK (nivel_final = ANY
    (ARRAY['sin_normalizar'::text, '1fn'::text, '2fn'::text, '3fn'::text])),
  esquema_original jsonb,
  esquema_final jsonb,
  dependencias jsonb,
  violaciones jsonb,
  sugerencias jsonb,
  CONSTRAINT historial_validaciones_pkey PRIMARY KEY (id),
  CONSTRAINT historial_validaciones_user_id_fkey FOREIGN KEY (user_id)
    REFERENCES public.usuarios(id)
);

CREATE INDEX IF NOT EXISTS idx_historial_validaciones_user_id
  ON public.historial_validaciones(user_id);
CREATE INDEX IF NOT EXISTS idx_historial_validaciones_fecha
  ON public.historial_validaciones(fecha);
CREATE INDEX IF NOT EXISTS idx_historial_validaciones_nivel_final
  ON public.historial_validaciones(nivel_final);
CREATE INDEX IF NOT EXISTS idx_presencia_ultima_actividad
  ON public.presencia(ultima_actividad);

```

## 7. Lenguaje de Manipulación de Datos (DML)

Los siguientes ejemplos DML utilizan datos académicos ficticios. No deben reemplazar contraseñas reales ni exponer información sensible. Se presentan como guía para pruebas de funcionamiento y sustentación del diccionario de datos.

```

-- 1. Insertar usuario académico ficticio
INSERT INTO public.usuarios (username, password_hash)
VALUES ('estudiante_dataquest', '$2b$12$hash_academico_de_prueba')
RETURNING id;

-- 2. Insertar perfil asociado
INSERT INTO public.perfiles (id, nombre, avatar_url)
SELECT id, 'Estudiante DataQuest', 'https://example.com/avatar.png'
FROM public.usuarios
WHERE username = 'estudiante_dataquest';

-- 3. Registrar presencia del usuario
INSERT INTO public.presencia (user_id, nombre)
SELECT id, 'Estudiante DataQuest'
FROM public.usuarios
WHERE username = 'estudiante_dataquest';

-- 4. Registrar historial de validación con JSONB
INSERT INTO public.historial_validaciones (
  user_id, nombre_esquema, formato_entrada, nivel_inicial, nivel_objetivo, nivel_final,
  esquema_original, esquema_final, dependencias, violaciones, sugerencias
)
SELECT
  id,
  'Esquema académico de ventas',
  'sql',
  'sin_normalizar',
  '3fn',
  '2fn',
  '{"tablas":[{"nombre":"venta","campos":["id","cliente","productos"]}]}':jsonb,
  '{"tablas":[{"nombre":"venta"}, {"nombre":"venta_detalle"}]}':jsonb,
  '{"determinante":"id","dependiente":"cliente"}':jsonb,
  '{"1fn":["Campo productos contiene valores múltiples"],"2fn":[],"3fn":[]}':jsonb,
  '{"nivel":"1fn","accion":"Separar productos en tabla detalle"}':jsonb
FROM public.usuarios
WHERE username = 'estudiante_dataquest';

-- 5. Consultar usuarios registrados
SELECT id, username, creado_en
FROM public.usuarios
ORDER BY creado_en DESC;

-- 6. Consultar perfiles

```

```

SELECT u.username, p.nombre, p.avatar_url
FROM public.usuarios u
JOIN public.perfiles p ON p.id = u.id;

-- 7. Consultar usuarios conectados o activos
SELECT p.nombre, pr.conectado_en, pr.ultima_actividad
FROM public.presencia pr
LEFT JOIN public.perfiles p ON p.id = pr.user_id
ORDER BY pr.ultima_actividad DESC;

-- 8. Consultar historial por usuario
SELECT hv.fecha, hv.nombre_esquema, hv.formato_entrada,
       hv.nivel_inicial, hv.nivel_objetivo, hv.nivel_final
FROM public.historial_validaciones hv
JOIN public.usuarios u ON u.id = hv.user_id
WHERE u.username = 'estudiante_dataquest'
ORDER BY hv.fecha DESC;

-- 9. Consultar validaciones por nivel final
SELECT nivel_final, COUNT(*) AS total
FROM public.historial_validaciones
GROUP BY nivel_final
ORDER BY total DESC;

-- 10. Consultar validaciones con violaciones registradas
SELECT id, nombre_esquema, violaciones
FROM public.historial_validaciones
WHERE violaciones IS NOT NULL
   AND violaciones::text <> '{}':text;

-- 11. Actualizar última actividad
UPDATE public.presencia
SET ultima_actividad = now()
WHERE user_id = (
  SELECT id FROM public.usuarios WHERE username = 'estudiante_dataquest'
);

-- 12. Actualizar avatar del perfil
UPDATE public.perfiles
SET avatar_url = 'https://example.com/avatar_actualizado.png'
WHERE id = (
  SELECT id FROM public.usuarios WHERE username = 'estudiante_dataquest'
);

-- 13. Eliminación controlada de una validación de prueba
DELETE FROM public.historial_validaciones
WHERE nombre_esquema = 'Esquema académico de ventas'
   AND user_id = (SELECT id FROM public.usuarios WHERE username = 'estudiante_dataquest');

```

## 8. Reglas de Integridad y Seguridad

| Tipo de regla             | Aplicación en DataQuest                                            | Detalle técnico                                                                                          |
|---------------------------|--------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| Integridad de entidad     | Todas las tablas implementadas tienen clave primaria UUID.         | usuarios.id, perfiles.id, presencia.id e historial_validaciones.id identifican registros únicos.         |
| Integridad referencial    | perfiles, presencia e historial_validaciones dependen de usuarios. | Las claves foráneas evitan asociar perfiles o validaciones a usuarios inexistentes.                      |
| Integridad de dominio     | Campos formato_entrada y niveles se controlan con CHECK.           | Solo se aceptan valores definidos como sql, csv, xls, txt, texto_pegado, sin_normalizar, 1fn, 2fn y 3fn. |
| Integridad de negocio     | El nombre del esquema es obligatorio.                              | historial_validaciones.nombre_esquema tiene restricción NOT NULL.                                        |
| Seguridad de credenciales | password_hash almacena hash y no contraseña plana.                 | Debe usarse hashing seguro desde la capa de aplicación.                                                  |
| Control de duplicidad     | username y presencia.user_id son únicos.                           | Evita cuentas duplicadas y múltiples registros de presencia por usuario.                                 |
| Auditoría recomendada     | No hay tabla de auditoría implementada.                            | Se recomienda implementar auditoria_eventos o triggers de auditoría.                                     |

## 9. Matriz de Trazabilidad de Datos

| Tabla    | Módulo que la usa      | Requerimiento asociado                | Endpoint / función asociada                           | Operaciones CRUD | Estado       | Evidencia esperada                |
|----------|------------------------|---------------------------------------|-------------------------------------------------------|------------------|--------------|-----------------------------------|
| usuarios | Autenticación / sesión | Registro e identificación de usuarios | Login / registro en capa Streamlit o controlador auth | CREATE, READ     | Implementada | Usuario registrado y autenticado. |

| Tabla                  | Módulo que la usa     | Requerimiento asociado                  | Endpoint / función asociada                                                                | Operaciones CRUD                     | Estado       | Evidencia esperada                            |
|------------------------|-----------------------|-----------------------------------------|--------------------------------------------------------------------------------------------|--------------------------------------|--------------|-----------------------------------------------|
| perfiles               | Perfil / comunidad    | Mostrar información pública del usuario | Controlador de perfil o comunidad                                                          | CREATE, READ, UPDATE                 | Implementada | Nombre y avatar visibles.                     |
| presencia              | Comunidad / actividad | Control de usuarios activos             | Actualización de presencia en tiempo real                                                  | CREATE, READ, UPDATE, DELETE         | Implementada | Última actividad actualizada.                 |
| historial_validaciones | Validador / historial | Guardar resultados de normalización     | guardar_historial / actualizar_historial / POST /api/normalizacion como fuente de análisis | CREATE, READ, UPDATE parcial, DELETE | Implementada | Validación guardada con JSONB de diagnóstico. |

## 10. Conclusiones Técnicas

- El diccionario de datos documenta formalmente el esquema real utilizado por DataQuest, evitando clasificar como implementadas tablas que no existen en la base proporcionada.
- La tabla usuarios funciona como entidad central para relacionar perfiles, presencia e historial de validaciones.
- La tabla historial\_validaciones concentra la evidencia académica del proceso de normalización y justifica el uso de JSONB para almacenar resultados flexibles.
- Las restricciones CHECK fortalecen la integridad de dominio al limitar formatos de entrada y niveles de normalización.
- El modelo actual es suficiente para una primera versión académica, aunque puede enriquecerse con auditoría, reportes y resultados detallados en futuras versiones.

## 11. Recomendaciones

- Mantener actualizado el archivo db/modelos.sql cada vez que se modifique el esquema físico.
- Sincronizar el diccionario de datos con FD03, FD05, FD06 y el repositorio del proyecto.
- Agregar migraciones controladas para evitar cambios manuales no documentados en Supabase/PostgreSQL.
- Implementar índices sobre campos consultados frecuentemente, especialmente user\_id, fecha, nivel\_final y ultima\_actividad.
- Mantener el almacenamiento de contraseñas únicamente como hash seguro y nunca en texto plano.
- Evaluar una tabla de auditoría para registrar eventos críticos como inserción, actualización o eliminación de validaciones.
- Considerar tablas normalizadas futuras si los reportes requieren consultas SQL más analíticas sobre dependencias, violaciones y sugerencias.
- Validar el DDL en un entorno de prueba antes de ejecutarlo en producción académica.